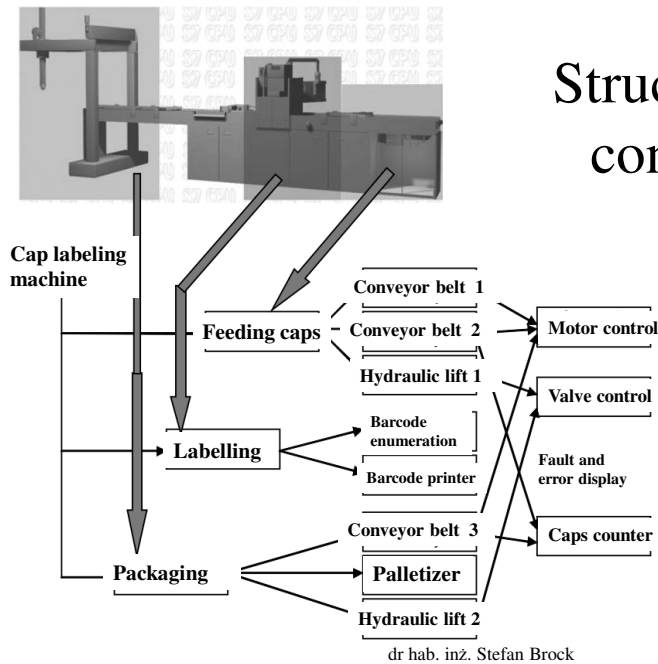
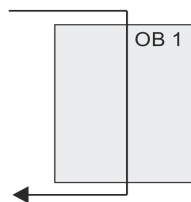




1

1

Linear organisation of the software



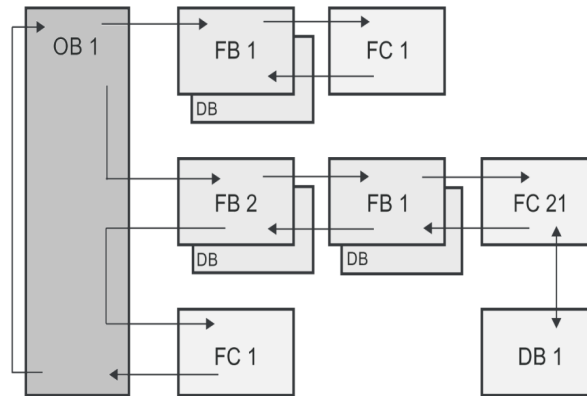
- the program is non-transparent and therefore sensitive to errors,
- the program code is long,
- running program fragments is uncomfortable,
- there is no support for group work,
- there is no support for the use of previously developed solutions in the project.

dr hab. inż. Stefan Brock

2

2

Modular organisation of the software



dr hab. inż. Stefan Brock

3

3

Modular organisation of the software

- easier and faster programming and significantly easier commissioning of complex control systems,
- The designer has a better insight into the entire control process,
- each block responsible for a separate function or system operation can be tested and maintained more easily in operation;
- updating and modifying the software code is easier and faster,
- once written and tested software blocks, such as the control of conveyors in different devices, can be reused,
- mechanisms of creating libraries of software modules are available, which significantly facilitate team work on the project,
- creating modular components linked to specific technological functions simplifies and accelerates the implementation of the entire application.

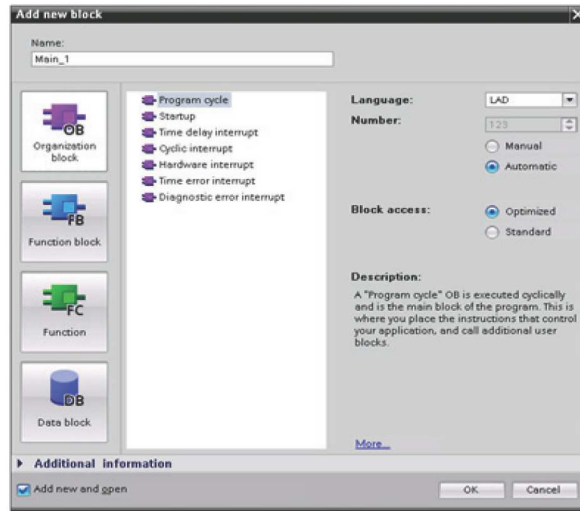
dr hab. inż. Stefan Brock

4

4

Software blocks in S7-1200

- Organization block
- Function block
- Function
- Data block



dr hab. inż. Stefan Brock

5

5

Organization block OB

Basic units of software organization with modular structure. Calling and execution of the OB block takes place after the occurrence of a certain logical condition in the controller. OB blocks cannot be called from other parts of the program.

- **Program cycle.** Cyclic OB blocks are called in each work cycle of the controller. Basic block OB 1, as well as subsequent cyclic blocks, executed successively after completion of OB 1.
- **Startup.** Starting blocks, called during system start-up or restart. Block OB 100 and further.
- **Time delay interrupt.** Delayed program blocks that will be called with a specified delay after the selected event occurs. The delay time is counted down in the system, and after the countdown, regardless of the current phase of the controller's work, an appropriate delayed OB block is called.

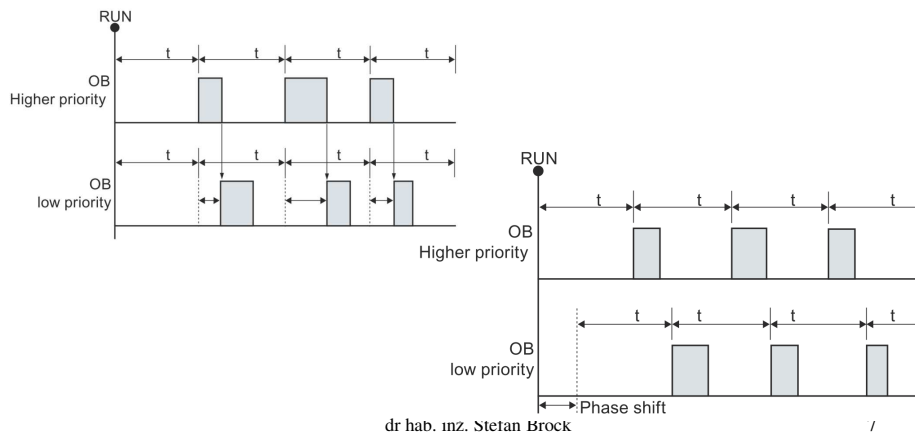
dr hab. inż. Stefan Brock

6

6

Organization block OB

- **Cyclic interrupt.** Isochronous blocks are called up at fixed, strictly defined intervals. It is possible to define several isochronous OB blocks with the same repetition period and individually defined relative delay (phase).



dr hab. inż. Stefan Brock

/

7

Organization block OB

Hardware interrupt. Hardware interrupt software blocks are associated with various external events. Each event can be assigned an OB block, and this assignment can be dynamically modified during program operation.

- signal change - a rising or falling edge on one of the digital inputs of the controller;
- reaching by the hardware fast counter of the setpoint value of the counted impulses;
- change of counting direction of the fast counter, which corresponds to the situation of return of an electric drive equipped with a quadrature sensor (with A/B channels) connected to the controller input;
- external fast counter reset, which corresponds to the situation where a reference pulse is generated by a quadrature sensor with a Z-channel.

dr hab. inż. Stefan Brock

8

8

Time error interrupt. OB number 80, triggered by exceeding the preset duty cycle and real time clock errors.

- If this block is not defined, the event of exceeding the maximum operating cycle causes the controller to switch to STOP mode.
- If the programmer defines OB 80 block, it should ensure safe operation of this event.

Diagnostic error interrupt. OB number 82, triggered by the interruption of a diagnostic system fault. Such errors are reported by the control modules as a result:

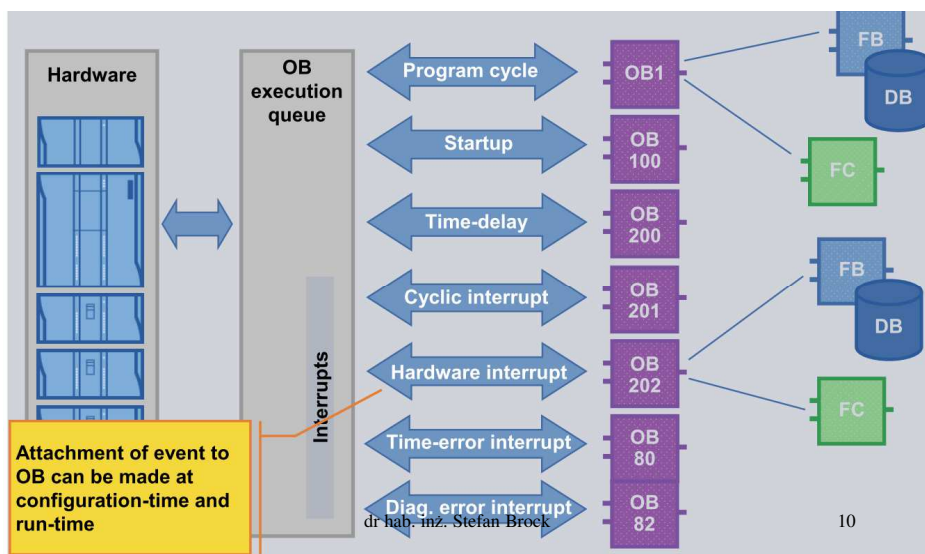
- short-circuits or interruptions in the wiring of analogue modules,
- power loss,
- exceeding the lower or upper processing limit.
- If the OB 82 block is not defined in the system, the controller ignores the diagnostic error message.
- If the OB 82 block is defined, the system designer may, for example, force the controller to stop working.

dr hab. inż. Stefan Brock

9

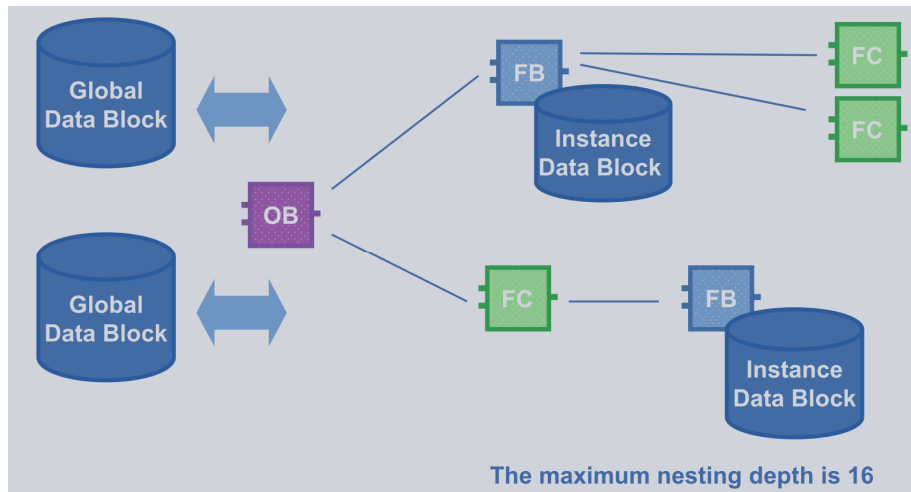
9

Event handling



10

Structure of a PLC program



dr hab. inż. Stefan Brock

11

11

Functions FC

Functions FC are program blocks that perform a certain task.

Functions are called from other program blocks - organisational blocks, function blocks or functions.

A function can have input variables that read values from the call point and output variables that transmit the assigned values to the call point.

The function can also use local variables, created dynamically when the function is called.

There are also available global variables (visible in the whole project) and it is possible to use absolute addressing of resources (input, output and memory area) of the controller.

Built-in functions such as, for example, mathematical functions - trigonometric functions, square root function and a number of other functions.

dr hab. inż. Stefan Brock

12

12

Function Blocks FB

The FB function blocks are similar to the FC function block.

The function block can have input, output, local variables, it can have access to global variables.

It also has static variables whose value is remembered between function block calls.

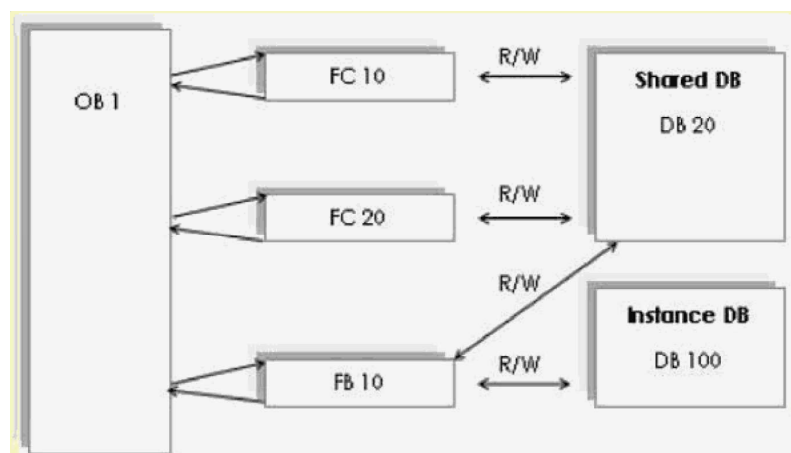
Each time a function block is used in a program, it creates a new data block (function block instance).

Concepts similar to calls in the program of counter blocks or timers.

dr hab. inż. Stefan Brock

13

13

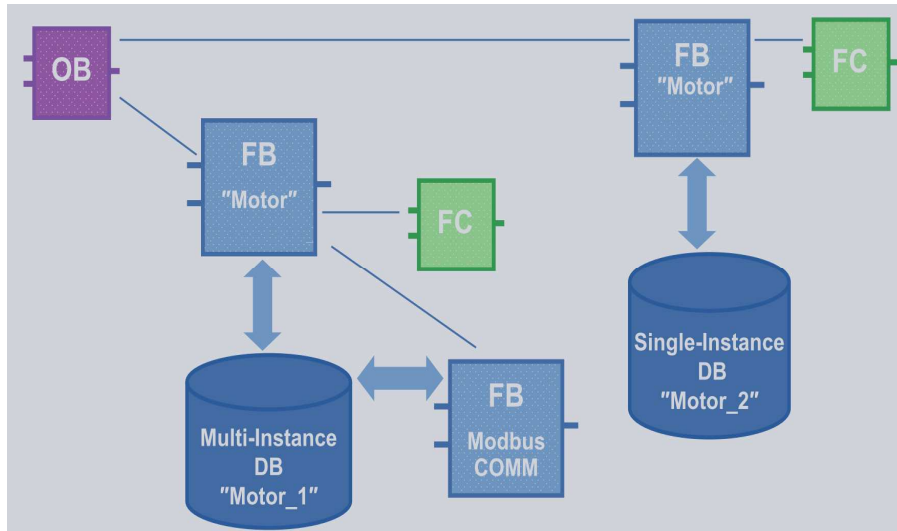


dr hab. inż. Stefan Brock

14

14

FC and FB calling



dr hab. inż. Stefan Brock

15

15

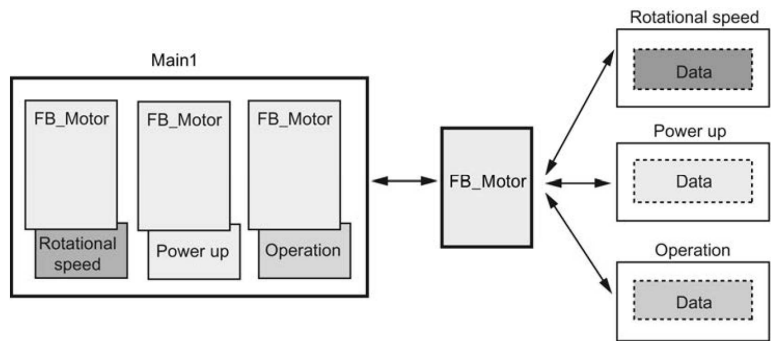
- The system defines a certain OB block which calls twice function blocks FB "Motor" used to control motors.
- Since the FB block "Motor" was used twice in the OB block code, two data blocks DB "Motor_1" and DB "Motor_2" were created, which contain static data related to the operation of two separate motors.
- The FC function, which performs certain calculations, for example, is called up from FB "Motor".
- Calling the FC function does not involve the creation of a data block. In addition, the "Motor_1" DB block has been assigned the multi-user attribute Multi-Instance - data block sharing between different function blocks.
- In addition to the function block FB "Motor", the function block for communication in the Modbus FB "Modbus COMM" network also has access to the DB "Motor_1" data block. This makes it possible to remotely monitor and control the function of the drive supported by this instance of the "Motor" FB block.

dr hab. inż. Stefan Brock

16

16

Example: single instances

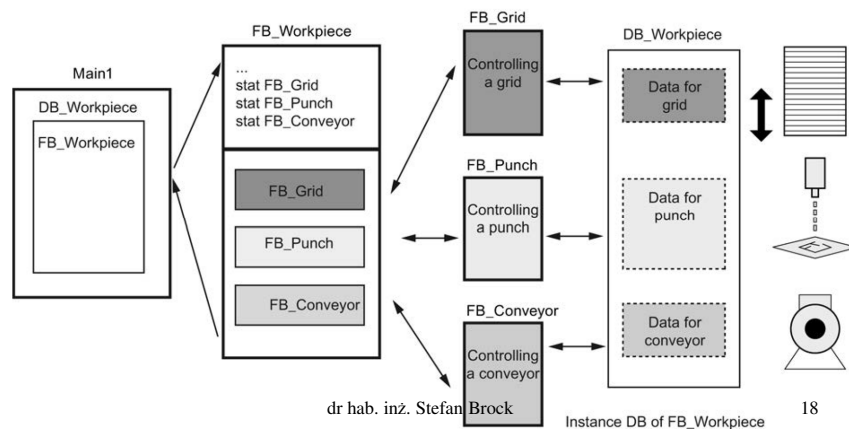


dr hab. inż. Stefan Brock

17

17

Example: Different function blocks share data from the calling block



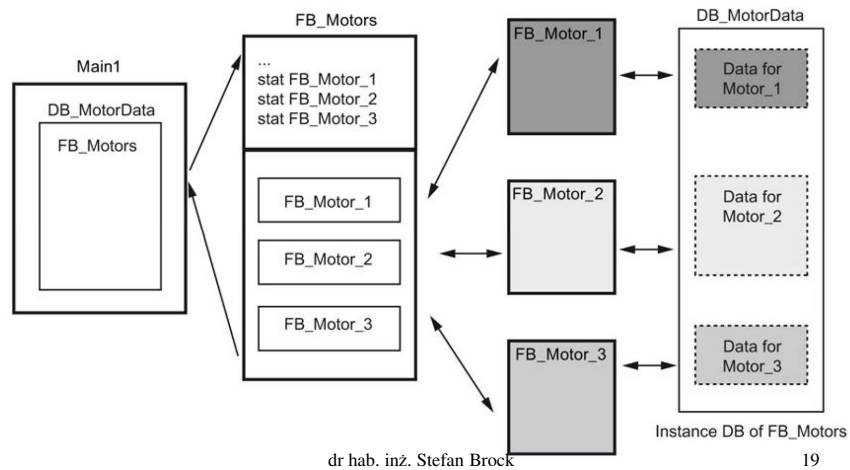
dr hab. inż. Stefan Brock

Instance DB of FB_Workpiece

18

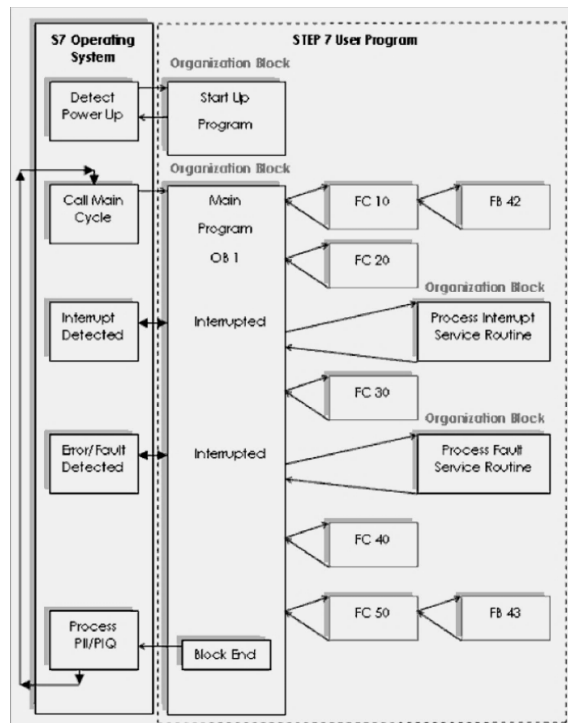
18

Example: Data blocks of called functions located in the data block of the calling function block



19

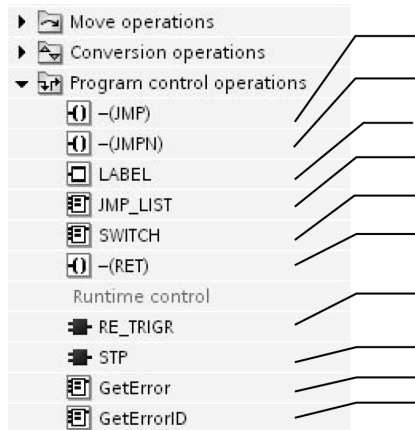
19



20

20

Instructions controlling the program flow

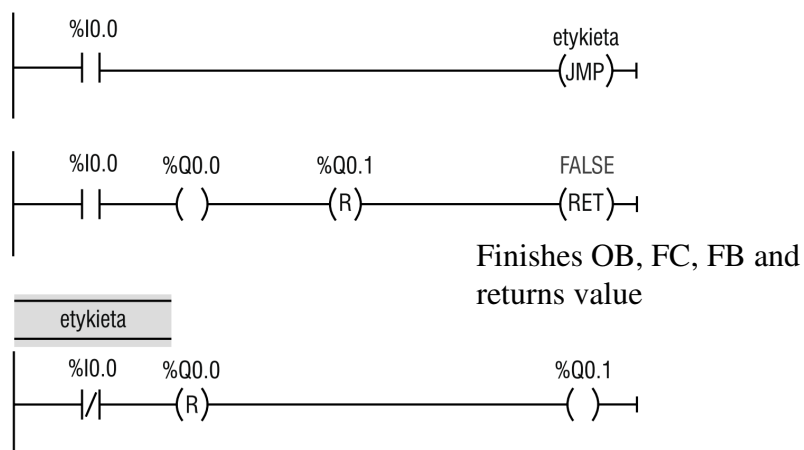


dr hab. inż. Stefan Brock

21

21

Jump to label

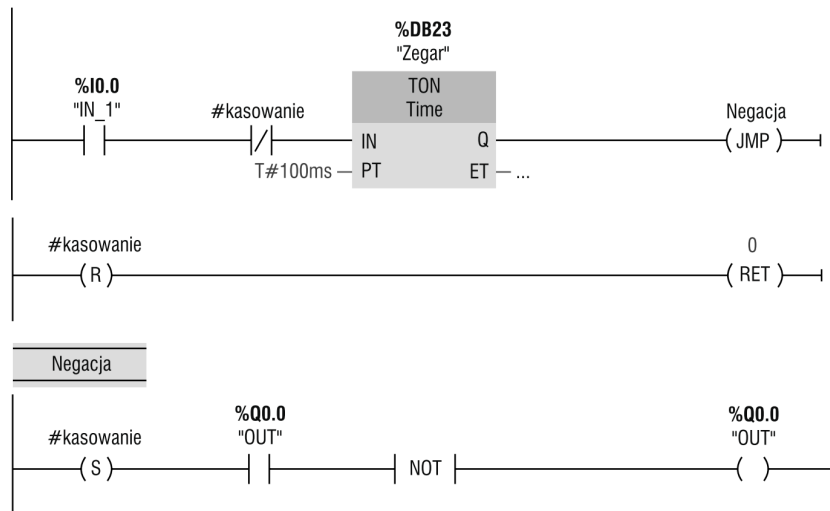


dr hab. inż. Stefan Brock

22

22

Jump to label



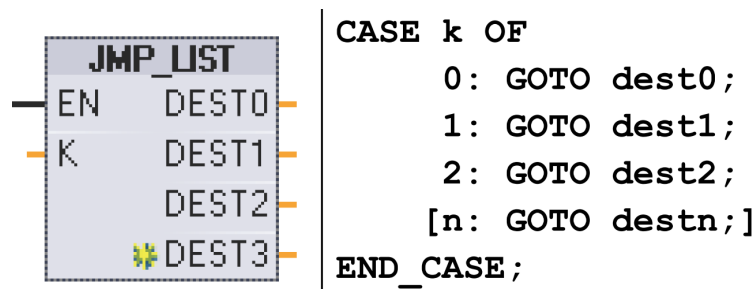
Wykorzystano ilustracje z książki:
J. Kwaśniewski „Sterowniki S7-1200
w praktyce inżynierskiej”

dr hab. inż. Stefan Brock

23

23

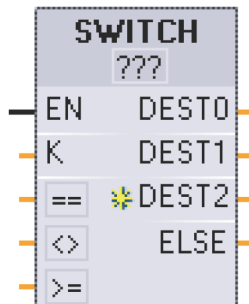
Jump to label from the list



dr hab. inż. Stefan Brock

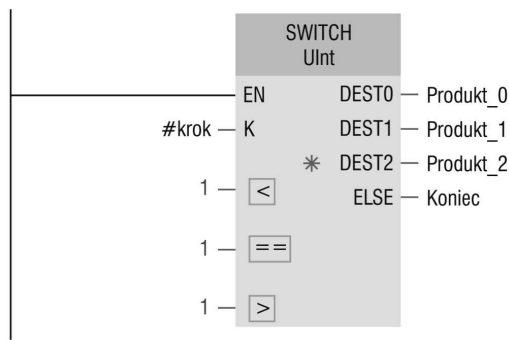
24

24



Switch

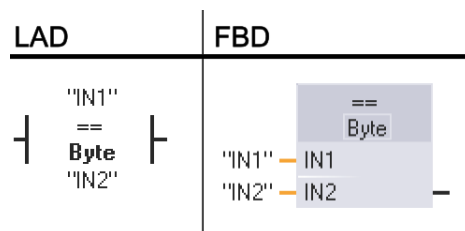
Jump to a label corresponding to the first true condition from the list.



25

25

Comparators



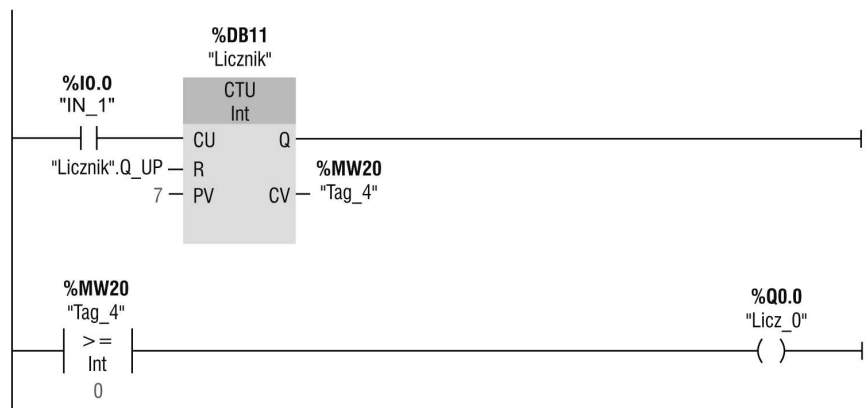
=	IN1 is equal to IN2
<>	IN1 is not equal to IN2
>=	IN1 is greater than or equal to IN2
<=	IN1 is less than or equal to IN2
>	IN1 is greater than IN2
<	IN1 is less than IN2

dr hab. inż. Stefan Brock

26

26

Comparators



Wykorzystano ilustracje z książki:
J. Kwaśniewski „Sterowniki S7-1200
w praktyce inżynierskiej”

dr hab. inż. Stefan Brock

27

27

„In range”, „Out Range”

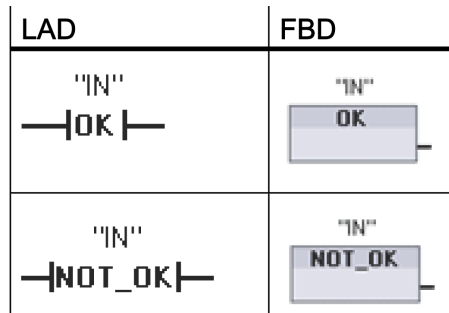
LAD / FBD	SCL
	<pre>out := IN_RANGE(min, val, max);</pre>
	<pre>out := OUT_RANGE(min, val, max);</pre>

dr hab. inż. Stefan Brock

28

28

Validity test of the real number



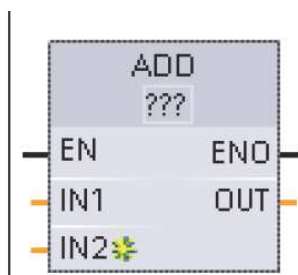
IN	Real, LReal
----	-------------

dr hab. inż. Stefan Brock

29

29

Mathematical operations: elementary



- ADD: Addition ($IN1 + IN2 = OUT$)
- SUB: Subtraction ($IN1 - IN2 = OUT$)
- MUL: Multiplication ($IN1 * IN2 = OUT$)
- DIV: Division ($IN1 / IN2 = OUT$)

IN1, IN2	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Constant
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal

dr hab. inż. Stefan Brock

30

30

Mathematical operations: elementary

Output ENO

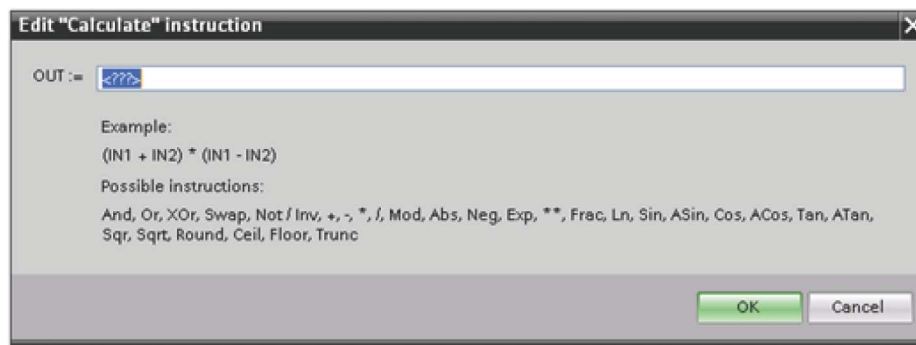
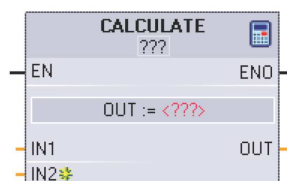
ENO	Description
1	No error
0	The Math operation result value would be outside the valid number range of the data type selected. The least significant part of the result that fits in the destination size is returned.
0	Division by 0 (IN2 = 0): The result is undefined and zero is returned.
0	Real/LReal: If one of the input values is NaN (not a number) then NaN is returned.
0	ADD Real/LReal: If both IN values are INF with different signs, this is an illegal operation and NaN is returned.
0	SUB Real/LReal: If both IN values are INF with the same sign, this is an illegal operation and NaN is returned.
0	MUL Real/LReal: If one IN value is zero and the other is INF, this is an illegal operation and NaN is returned.
0	DIV Real/LReal: If both IN values are zero or INF, this is an illegal operation and NaN is returned.

dr hab. inż. Stefan Brock

31

31

Mathematical operations: complex



32

Mathematical operations: functions

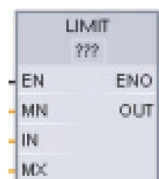
MOD	MOD (Return remind of division)
NEG	NEGation – zmiana znaku argume
INC	INCrement – inkrementacja argun
DEC	DECrement – dekrementacja argu
ABS	ABSolutvalue – obliczanie wartoś
MIN	MIN (get minimum) – wybór mnie
MAX	MAX (get maximum) – wybór wię
LIMIT	LIMIT (set limit value) – sprawdze
SQR	SQR (square) – podnoszenie do k
SQRT	SQRT (square root) – pierwiastek
LN	LN (natural logarithm) – logarytm
EXP	EXPonential value – funkcja wykład
SIN	SIN (sine value) – sinus kąta (w r
COS	COS (cosine value) – cosinus ką
TAN	TAN (tangent value) – tangens ką
ASIN	ASIN (arcsine value) – arcus sinu
ACOS	ACOS (arccosine value) – arcus c
ATAN	ATAN (arctangent value) – arcus t
FRAC	FRACTION – zwraca część ułamko
EXPT	EXPT (exponentiate) – funkcja wyl

33

33

Limit

LAD / FBD



SCL

```
LIMIT(MN:=_variant_in_,
      IN:=_variant_in_,
      MX:=_variant_in_,
      OUT:=_variant_out_);
```

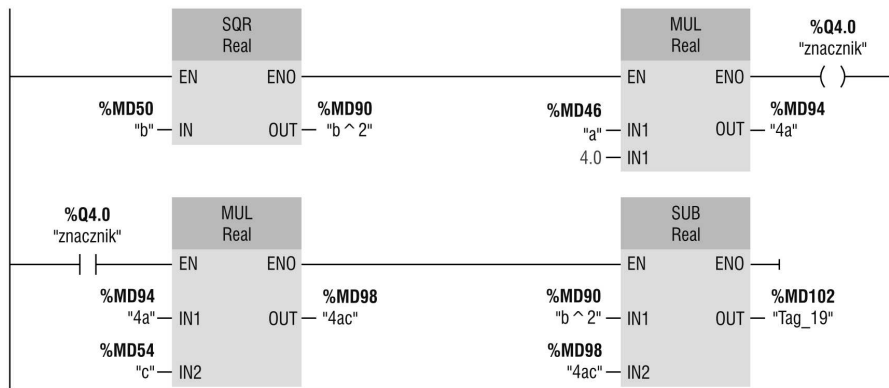
$$OUT = \begin{cases} MN & \text{if } IN < MN \\ IN & \text{if } MN \leq IN \leq MX \\ MX & \text{if } IN > MX \end{cases}$$

dr hab. inż. Stefan Brock

34

34

Example - square equation

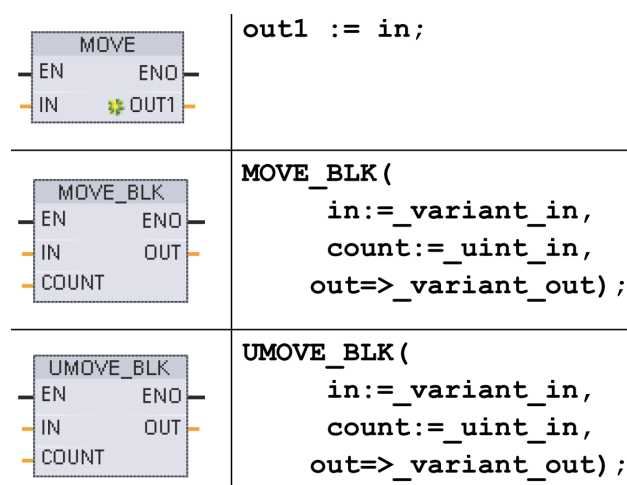


dr hab. inż. Stefan Brock

35

35

Move instructions

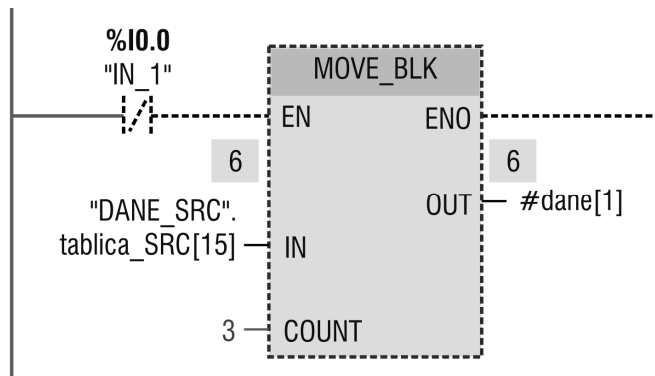


dr hab. inż. Stefan Brock

36

36

Copying a part of the array

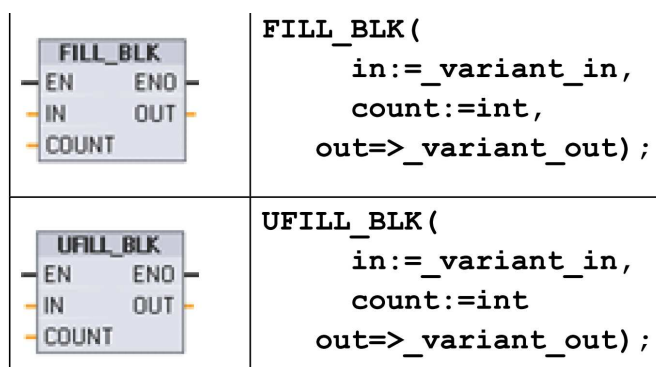


dr hab. inż. Stefan Brock

37

37

Instructions for filling in areas

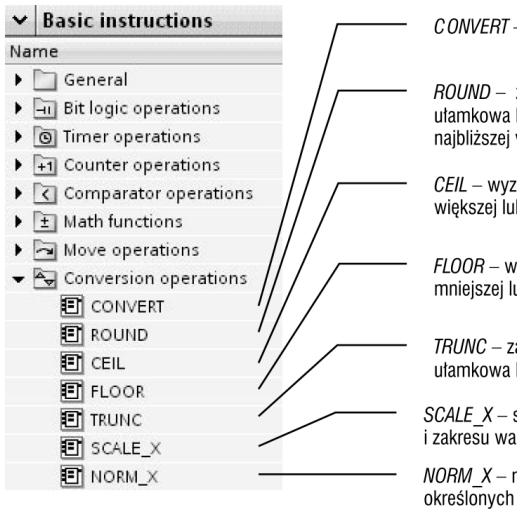


dr hab. inż. Stefan Brock

38

38

Conversion operations



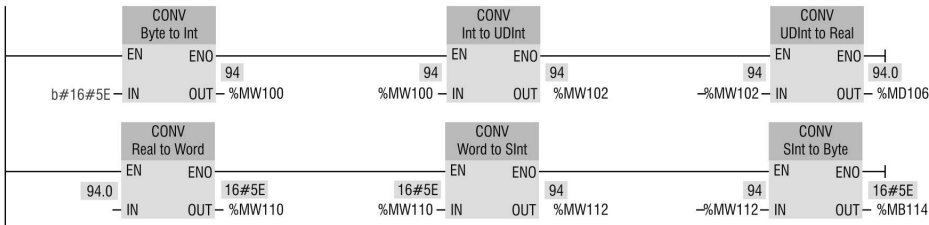
dr hab. inż. Stefan Brock

39

39

Type conversion

LAD / FBD	SCL
	<code>out := <data type in>_TO_<data type out>(in);</code>

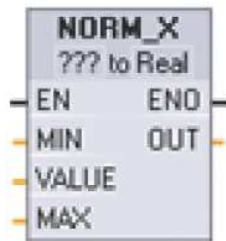


dr hab. inż. Stefan Brock

40

40

Normalisation



```
out :=NORM_X(min:=_in_,
              value:=_in_,
              max:=_in_);
```

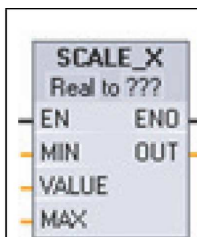
$OUT = (VALUE - MIN) / (MAX - MIN),$
 where ($0.0 \leq OUT \leq 1.0$)

dr hab. inż. Stefan Brock

41

41

Scale - denormalisation



```
out :=SCALE_X(min:=_in_,
               value:=_in_,
               max:=_in_);
```

$0.0 \leq VALUE \leq 1.0$

$OUT = VALUE (MAX - MIN) + MIN$

dr hab. inż. Stefan Brock

42

42